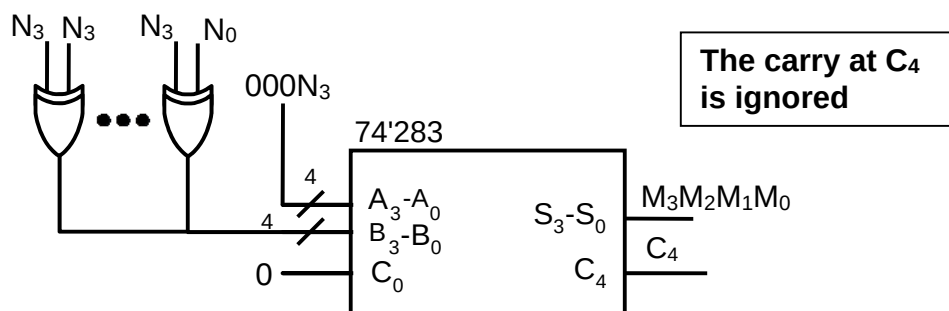


EE2020 Tutorial 5 - Solutions

1.

We are required to find the magnitude, M , of a 4-bit number, N , expressed in 2's complement notation. A 4-bit parallel adder (74'283) is used.

In a circuit implementation, the magnitude of a 2's complement is found by inverting all the bits of a number and adding 1 to it if the number is negative. We can equivalently write $M = N$ if N is positive. Else $M = \text{complement}(N) + 1$ if N is negative (i.e., $B=N$ if $N_3=0$, $B=\text{NOT}(N)$ if $N_3=1$; both conditions are achieved by summing $000N_3$).



The dataflow Verilog description of the intended function (magnitude of a number expressed in 2's complement representation) can be written by first considering that the magnitude of a number expressed in 2's complement representation is the number itself if it is positive (i.e., if the MSB $N[3]$ is equal to 0), and its 2's complement if it is negative (i.e., equal to the bit-wise complement plus one, if $N[3]=1$). See solution of Tutorial 2 (week 3) for the details.

Hence, M needs to be conditionally assigned to N if $N[3]=0$, and $\sim N+1$ if $N[3]=1$. This can be done by using the conditional operator "?" in a continuous assignment, which has the following syntax: *cond_expr* ? *true_expr* : *false_expr*, where

- *cond_expr* is the expression that determines the condition (it is $N[0]$ in this case, which can be either 1 or 0)
- *true_expr* is the assignment when *cond_expr* = 1 (i.e., $N[3]=1$ in this case)
- *false_expr* is the assignment when *cond_expr* = 0 (i.e., $N[3]=0$ in this case)

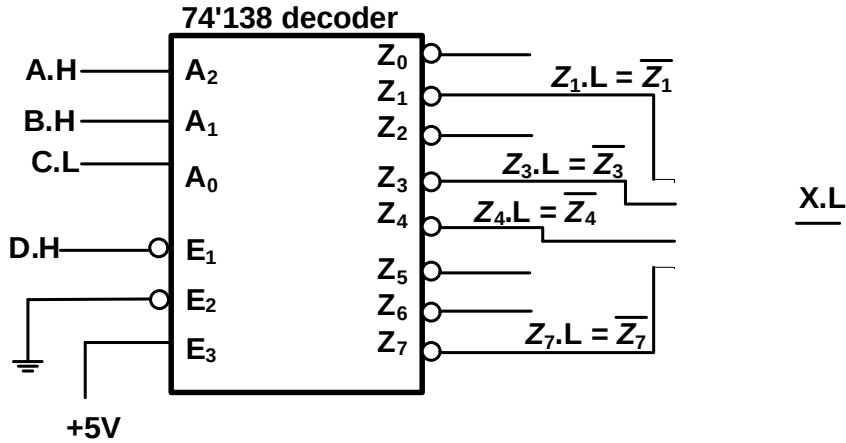
The resulting Verilog code is:

```

module func(M,N);
    input [3:0] N;
    output [3:0] M;
    assign M = N[3] ? (~N+1) : N; // continuous assignment using ? operator
    // meaning: if N[3]=1, M is set at the 2's complement of N (bit-wise NOT + 1)
    // if N[3]=0, M is set at N
endmodule

```

2. A 3-to-8 decoder (74'183) is here used along with a physical NAND gate. In the following, the “*.H” notation will be omitted for active-high signals to make notation more compact, whereas only the “*.L” notation will be made explicit for active-low signals.



E_1 and E_2 are active-low enable signals, E_3 is active-high enable signal (i.e., the outputs of the decoder can be asserted only if $E_1=E_2=0$ and $E_3=1$, i.e. if $E_3 \cdot \overline{E_2} \cdot \overline{E_1} = 1$).

The primary inputs of the decoder are $A_2 = A$, $A_1 = B$, $A_0 = C.L = \bar{C}$, $E_3 = 1$, $E_2.L = 0$ (i.e., E_2 is constantly asserted), $E_1.L = D$ (i.e., $E_1 = \bar{D}$). Then, from inspection of the NAND gate, the output is

$$X = \overline{\overline{Z_1} \cdot \overline{Z_3} \cdot \overline{Z_4} \cdot \overline{Z_7}}$$

but actually the output is active low (X.L), hence

$$X.L = \bar{X} = \overline{\overline{Z_1} \cdot \overline{Z_3} \cdot \overline{Z_4} \cdot \overline{Z_7}}$$

where $\overline{Z_1}$, $\overline{Z_3}$, $\overline{Z_4}$ and $\overline{Z_7}$ are known from the understanding of the decoder behavior:

$$\overline{Z_1} = Z_1.L = \overline{A_2 \cdot A_1 \cdot A_0 \cdot E_3 \cdot \overline{E_2} \cdot \overline{E_1}} = \bar{A} \cdot \bar{B} \cdot \bar{C} \cdot 1 \cdot 1 \cdot \bar{D} = \bar{A} \cdot \bar{B} \cdot \bar{C} \cdot \bar{D}$$

$$\overline{Z_3} = \overline{A_2 \cdot A_1 \cdot A_0 \cdot E_3 \cdot \overline{E_2} \cdot \overline{E_1}} = \bar{A} \cdot B \cdot \bar{C} \cdot \bar{D}$$

$$\overline{Z_4} = \overline{A_2 \cdot A_1 \cdot \bar{A_0} \cdot E_3 \cdot \overline{E_2} \cdot \overline{E_1}} = A \cdot \bar{B} \cdot C \cdot \bar{D}$$

$$\overline{Z_7} = \overline{A_2 \cdot A_1 \cdot A_0 \cdot E_3 \cdot \overline{E_2} \cdot \overline{E_1}} = A \cdot B \cdot \bar{C} \cdot \bar{D}$$

Combining the above results, we obtain

$$X.L = \overline{\overline{Z_1} \cdot \overline{Z_3} \cdot \overline{Z_4} \cdot \overline{Z_7}} = \overline{\bar{A} \cdot \bar{B} \cdot \bar{C} \cdot \bar{D} \cdot \bar{A} \cdot B \cdot \bar{C} \cdot \bar{D} \cdot A \cdot \bar{B} \cdot C \cdot \bar{D} \cdot A \cdot B \cdot \bar{C} \cdot \bar{D}}$$

which can be rearranged in several ways by using the De Morgan's laws (for example, you can complement twice, and then distribute the lower complement across the four terms in the product – try yourself).

3. a) Table I

S ₃	S ₂	S ₁	S ₀	Value (decimal representation of 2's complement)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	-8
1	0	0	1	-7
1	0	1	0	-6
1	0	1	1	-5
1	1	0	0	-4
1	1	0	1	-3
1	1	1	0	-2
1	1	1	1	-1

b)

Table II: Modifications to comparator output for signed comparison

Sign Bits		A > B Output	A = B Output	A < B Output
A ₃	B ₃			
0	0	No change	No change	No change
0	1	Invert output	No change	Invert output
1	0	Invert output	No change	Invert output
1	1	No change	No change	No change

After we write down Table I, it is clear that if A and B are of the same sign, then the magnitude comparator will give the correct outputs. However if A and B are of opposite sign, then the magnitude comparator outputs will be incorrect. It is easy to see from Table II that if A and B are of opposite sign we need to invert the A>B and A<B outputs of the magnitude comparator. An EX-OR gate can be used to detect whether A and B are of opposite sign. The output of the EX-OR gate can be used to invert the comparator outputs as appropriate.

OUTPUT(A=B)

output(A=B) always gives correct result

OUTPUT(A>B), OUTPUT(A<B)

0 0: ++ => all outputs need to be kept as they are

0 1: +- => output(A>B) should be 1, but in the above table it would be 0 (since neg. no. has always binary representation greater than pos. no.)
=> invert output(A>B) and output(A<B)

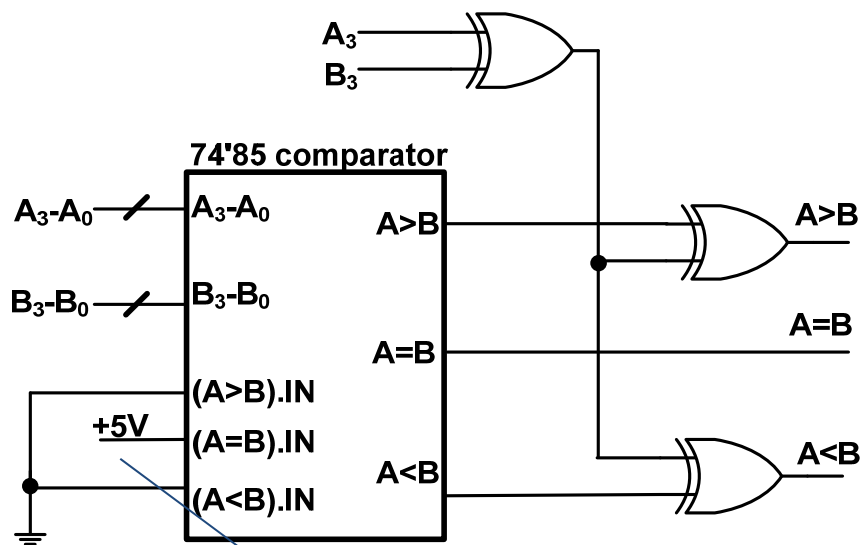
1 0: -+ => output(A>B) should be 0, but in the above table it would be 1 (since a negative number always has binary representation greater than pos. no.)
=> invert output(A>B) and output(A<B)

1 1: -- => since 2's compl representation of a negative number is higher for larger (less negative) number
=> keep output(A>B) and output(A<B) as they are

Voltage table of 74'85

Voltage Table

Comparing Inputs				Cascading Inputs			Outputs		
A ₃ , B ₃	A ₂ , B ₂	A ₁ , B ₁	A ₀ , B ₀	(A>B).I N	(A<B).I N	(A=B).I N	(A>B)	(A<B)	(A=B)
A ₃ >B ₃	X	X	X	X	X	X	H	L	L
A ₃ <B ₃	X	X	X	X	X	X	L	H	L
A ₃ =B ₃	A ₂ >B ₂	X	X	X	X	X	H	L	L
A ₃ =B ₃	A ₂ <B ₂	X	X	X	X	X	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ >B ₁	X	X	X	X	H	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ <B ₁	X	X	X	X	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ >B ₀	X	X	X	H	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ <B ₀	X	X	X	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	H	L	L	H	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	L	H	L	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	X	X	H	L	L	H
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	H	H	L	L	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	L	L	L	H	H	L



Cascading inputs (they represent lower-order bits) are configured so that they do not affect any output. Since they affect output only if inputs are equal, we want to generate output as if they are equal => (A>B).IN = 0, (A=B).IN = 1, (A<B).IN = 0.

Based on the above observations, and using the above discussed conditional operator, the resulting Verilog code is:

```
module twoscomplement_comparator(AgreaterB, AlowerB, AequalB,A,B);
    input [3:0] A, B;
```

```

output AgreaterB, AlowerB, AequalB;
wire samesign; // samesign is defined as the condition whether A and B have same sign
// (samesign=1 if A and B have same sign, samesign=0 if A and B have different sign)

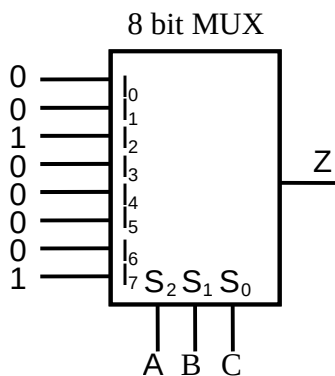
assign samesign = (A[3] == B[3]);
assign AgreaterB = samesign ? (A>B) : (A<B);
assign AequalB = (A == B);
assign AlowerB = samesign ? (A<B) : (A>B);
endmodule

```

4.

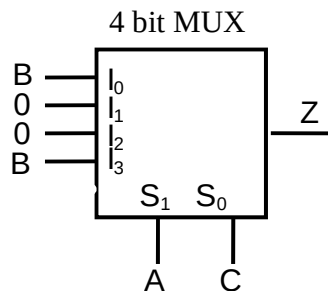
(a)

Let (A, B, C) be the 3-bit input. Then with connections as shown, $Z = B$ when $A=C$ else $Z = 0$.



(b)

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



If we connect A and C to S_1 and S_0 , respectively, in a 4-bit multiplexer then it is obvious from the problem statement (or the truth table) that if the connection to I_0 - I_3 are as shown, the function can indeed be implemented with a 4-bit multiplexer.

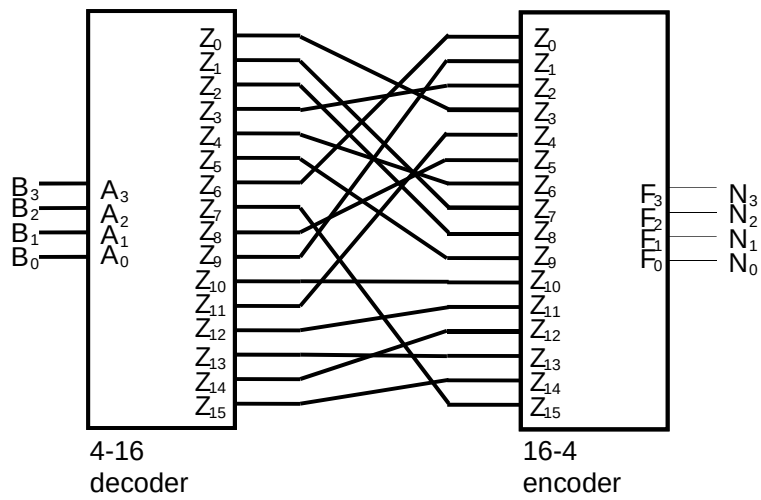
Based on the above observations, the resulting Verilog code is:

```

module func2(Z,A,B,C);
    input A, B, C;
    output Z;
    assign Z = (A == C) ? B : 0;
endmodule

```

5.



Decoder outputs the code number corresponding to the code that was present at its input. When these are appropriately connected to encoder inputs (as specified by table), the decoder outputs the corresponding N-code.